

---

## Módulo 1

# Preparación de datos con Python

---

*Análisis y Pronóstico de Datos Mineros con Python*

Apuntes + notebook de Google Colab

## Objetivo del módulo

---

Tomar una base de un proceso industrial y dejarla lista para el análisis temporal:

- cargarla correctamente (formato, separador, decimal, codificación);
- inspeccionar su estructura y sus tipos;
- corregir problemas básicos de calidad;
- organizar la variable temporal;
- hacer una primera exploración gráfica.

**Datos originales → datos limpios y estructurados**

## No se trata de aprender Python de memoria

---

En el curso se trabaja con **notebooks preparados en Google Colab**. Lo importante:

- reconocer la estructura del código;
- ejecutar las celdas **de arriba hacia abajo**;
- identificar qué variables hay que modificar;
- interpretar las salidas;
- adaptar el análisis a una nueva base.

carga → limpieza → análisis → modelo

Si una celda anterior no se ejecutó, las siguientes fallan.

## Las librerías de las primeras sesiones

---

### **pandas**

datos tabulares:  
importar, filtrar,  
fechas, faltantes,  
agrupar.

### **NumPy**

operaciones  
numéricas y valores  
especiales (`np.nan`).

### **matplotlib**

gráficos: `plot`, `hist`,  
`scatter`, `boxplot`.

### **seaborn**

se apoya en  
`matplotlib`; gráficos  
estadísticos en una  
línea.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns           # gráficos estadísticos, sobre matplotlib
```

No necesitamos cada librería completa: solo las herramientas del problema.

## La estructura de una base: el DataFrame

| Fecha            | Tonelaje_tph | Ley_Cu | Recuperacion_pct | Potencia_kW |
|------------------|--------------|--------|------------------|-------------|
| 01-08-2026 08:00 | 1450         | 0.82   | 88.2             | 3150        |
| 01-08-2026 09:00 | 1472         | 0.80   | 87.9             | 3180        |
| 01-08-2026 10:00 | 1435         | 0.85   | 89.1             | 3105        |

- cada **columna** es una variable; cada **fila**, una observación;
- variables típicas: tonelaje, flujo, presión, temperatura, ley, recuperación, potencia...
- y una variable especial:

**el tiempo**

## Importación: los detalles importan

---

```
df = pd.read_csv("datos.csv")           # caso simple
df = pd.read_csv("datos.csv", sep=";")   # export en español
df = pd.read_csv("datos.csv", sep=";", decimal=",") # 1450,5
df = pd.read_csv("datos.csv", encoding="latin1") # acentos, enie
df = pd.read_excel("datos.xlsx", sheet_name="Produccion")
```

**Ojo:** nunca asumir que una columna es numérica solo porque se ve numérica. Si el decimal es coma y no lo indicamos, pandas la lee como texto.

## Primera inspección: auditar antes de analizar

---

```
df.head()      df.tail()      df.shape      # (8760, 8)
df.columns     df.dtypes     df.info()
df.describe()
```

- `info()`: filas, columnas, no nulos, tipos;
- nombres con espacios: `df.columns = df.columns.str.strip();`
- `describe()` revela lo sospechoso: Tonelaje mínimo = -999.

**Ojo:** un -999 puede ser un código de "sin medición". Todavía no lo elimines: primero entiende qué significa.

## El tiempo como variable

---

Aunque veamos fechas, para Python Fecha suele ser object (texto).

```
df["Fecha"] = pd.to_datetime(df["Fecha"], dayfirst=True)
df = df.sort_values("Fecha")      # el orden contiene información
df = df.set_index("Fecha")        # DatetimeIndex
df.loc["2026-08-01"]              # selección por día
```

en una serie temporal, el orden de las observaciones no puede ignorarse



## Frecuencia de muestreo y huecos

---

```
df = df.asfreq("h")           # impone paso horario; los huecos aparecen como NaN
df.resample("D").mean()       # horario -> diario (media)
df.resample("D").sum()        # producción total diaria
```

- `asfreq()`: cambia/impone la frecuencia *sin* agregar;
- `resample()`: agrupa y necesita una regla (mean, sum, max...);
- la agregación correcta **depende de la variable**.

## Valores faltantes: limpiar $\neq$ inventar

---

```
df.isna().sum()
100 * df.isna().mean()
df.dropna()                # eliminar
df["Tonelaje"].interpolate() # interpolar
# ...o mantener el NaN y documentarlo
```

La decisión depende de: cantidad, duración del hueco, variable, origen y finalidad.

**No es lo mismo interpolar una hora perdida que reconstruir una detención de planta**

## Duplicados y timestamps repetidos

---

```
df.duplicated().sum()  
df = df.drop_duplicates()  
df.index.duplicated().sum()      # dos lecturas para la misma hora
```

Un timestamp repetido puede ser: dos mediciones legítimas, un duplicado, dos sensores, o algo a promediar. **Ojo:** no se decide solo con Python: primero hay que saber cómo se generaron los datos.

## Valores inconsistentes

---

Una observación puede existir y no ser NaN, y aun así ser incorrecta:

- Tonelaje = -999, Recuperación = 240%, Temperatura = -500 °C;
- un valor **constante** durante muchas horas  $\Rightarrow$  sensor congelado;
- cambios de escala, de unidades, saltos por recalibración.

dato improbable  $\neq$  dato incorrecto

Si eliminamos todo lo inusual, borramos justo los eventos que luego queremos detectar.

## Reemplazo de códigos conocidos

---

Si la documentación del sistema dice que -999 significa "sin medición":

```
| df["Tonelaje"] = df["Tonelaje"].replace(-999, np.nan)
```

Ahora Python sabe **explícitamente** que la información está ausente, en lugar de tratar -999 como un número real que distorsiona media y desviación.

## Los cuatro gráficos y sus preguntas

---

| Gráfico        | Pregunta principal                 |
|----------------|------------------------------------|
| Serie temporal | ¿Cómo evoluciona la variable?      |
| Histograma     | ¿Cómo se distribuyen sus valores?  |
| Boxplot        | ¿Dispersión y valores atípicos?    |
| Dispersión     | ¿Cómo se relacionan dos variables? |

El histograma **elimina el orden temporal**: dos series muy distintas pueden tener el mismo histograma. Esa idea será central al pasar a series temporales.

## Los cuatro gráficos: matplotlib y seaborn

---

La misma figura, dos formas de pedirla. seaborn recibe el DataFrame y los nombres de columnas; el eje y `plt.show()` siguen siendo de matplotlib.

```
# --- matplotlib ---
df["Tonelaje_tph"].plot()
df["Tonelaje_tph"].hist(bins=40)
plt.boxplot(df["Tonelaje_tph"].dropna())
plt.scatter(df["Tonelaje_tph"], df["Potencia_kW"])

# --- seaborn ---
sns.lineplot(data=df["Tonelaje_tph"])
sns.histplot(df["Tonelaje_tph"], bins=40)
sns.boxplot(y=df["Tonelaje_tph"])
sns.scatterplot(data=df, x="Tonelaje_tph", y="Potencia_kW")
```

# Flujo mínimo del Módulo 1

---

```
df = pd.read_csv("datos_proceso.csv", sep=";", decimal=",")
df.info(); df.describe()
df["Tonelaje"] = df["Tonelaje"].replace(-999, np.nan)
df["Fecha"] = pd.to_datetime(df["Fecha"], dayfirst=True)
df = df.sort_values("Fecha").drop_duplicates().set_index("Fecha")
df = df.asfreq("h")
df.isna().sum()
df["Tonelaje"].plot(figsize=(12, 4))
```



- asumir tipo numérico sin verificar dtypes;
- modelar sin haber mirado la serie;
- eliminar automáticamente todo valor extraño;
- interpolar un hueco largo como si fuera una medición aislada;
- olvidar `sort_values` antes de trabajar la serie.

## Qué deberías recordar

1. Siempre inspeccionar antes de analizar.
2. Verificar los tipos de las variables.
3. Una fecha en texto todavía no es una variable temporal.
4. En series temporales el orden importa.
5. Revisar frecuencia de muestreo y huecos.
6. Faltante, duplicado y anómalo son problemas *distintos*.
7. No eliminar automáticamente lo extraño.
8. Antes de modelar, visualizar.

La calidad del modelo nunca supera la calidad de los datos

El Módulo 1 termina con una base cargada, inspeccionada, con el tiempo ordenado, problemas reconocidos y una visualización preliminar.

M1: *¿mis datos están bien preparados?*

→ M2: *¿qué información estadística puedo extraer?*